

JOBSHEET 10

QUEUE



Nama : Fadila Mayamasita

NIM : 254107020012

Kelas : 1C Teknik Informatika

Program Studi D-IV Teknik Informatika

Jurusan Teknologi Informasi

Politeknik Negeri Malang 2026

2. Praktikum 1

2.1 Percobaan 1 : Operasi Dasar Queue

Waktu percobaan : 90 menit

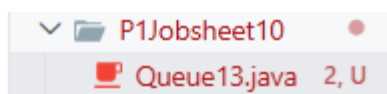
Pada percobaan ini, kita akan menerapkan operasi dasar pada algoritma Queue. Perhatikan Diagram Class Queue berikut ini:

Queue
data: int[] front: int rear: int size: int max: int
Queue(n: int) isFull(): boolean isEmpty(): boolean enqueue(dt: int): void dequeue(): int peek: void print(): void clear(): void

2.1.1. Langkah-langkah Percobaan

1. Buat folder baru bernama P1Jobsheet10 di dalam repository Praktikum ASD, kemudian buat class baru dengan nama Queue.

Jawaban :



2. Tambahkan atribut-atribut Queue sesuai diagram class, kemudian tambahkan pula konstruktornya seperti berikut ini :

Jawaban :

```
3 public class Queue13 {  
4     int[] data;  
5     int front;  
6     int rear;  
7     int size;  
8     int max;  
9  
10    public Queue13(int n) {  
11        max = n;  
12        data = new int[max];  
13        size = 0;  
14        front = rear = -1;  
15    }  
16 }
```

3. Buat method isEmpty bertipe boolean yang digunakan untuk mengecek apakah queue kosong.

Jawaban :

```
16
17     public boolean isEmpty() {
18         if (size == 0) {
19             return true;
20         } else {
21             return false;
22         }
23     }
```

4. Buat method IsFull bertipe boolean yang digunakan untuk mengecek apakah queue sudah penuh.

Jawaban :

```
24
25     public boolean isFull() {
26         if (size == 0) {
27             return true;
28         } else {
29             return false;
30         }
31     }
```

5. Buat method peek bertipe void untuk menampilkan elemen queue pada posisi paling depan.

Jawabna :

```
33     public void peek() {
34         if (!isEmpty()) {
35             System.out.println("Elemen terdepan: " + data[front]);
36         } else {
37             System.out.println(x: "Queue masih kosong");
38         }
39     }
```

6. Buat method print bertipe void untuk menampilkan seluruh elemen pada queue mulai dari posisi front sampai dengan posisi rear.

Jawaban :

```

40
41 public void print() {
42     if (isEmpty()) {
43         System.out.println(x: "Queue masih kosong");
44     } else {
45         int i = front;
46         System.out.print(data[i] + " ");
47         i = (i + 1) % max;
48     }
49     System.out.println(data[i] + " ");
50     System.out.println("Jumlah elemen = " + size);
51 }

```

7. Buat method clear bertipe void untuk menghapus semua elemen pada queue

Jawaban :

```

public void clear() {
    if (!isEmpty()) {
        front = rear = -1;
        size = 0;
        System.out.println(x: "Queue berhasil dikosongkan");
    } else {
        System.out.println(x: "Queue masih kosong");
    }
}

```

8. Buat method Enqueue bertipe void untuk menambahkan isi queue dengan parameter dt yang bertipe integer

Jawaban :

```

public void enqueue(int dt) {
    if (isFull()) {
        System.out.println(x: "Queue sudah penuh");
    } else {
        if (isEmpty()) {
            front = rear = 0;
        } else {
            if (rear == max - 1) {
                rear = 0;
            } else {
                rear++;
            }
        }
        data[rear] = dt;
        size++;
    }
}

```

9. Buat method Dequeue bertipe int untuk mengeluarkan data pada queue di posisi belakang

Jawaban :

```
81 public int Dequeue() {
82     int dt = 0;
83     if (isEmpty()) {
84         System.out.println(x: "Queue masih kosong");
85     } else {
86         dt = data[front];
87         size--;
88         if (isEmpty()) {
89             front = rear = -1;
90         } else {
91             if (front == max - 1) {
92                 front = 0;
93             } else {
94                 front++;
95             }
96         }
97     }
98     return dt;
99 }
```

10. Selanjutnya, buat class baru dengan nama QueueMain tetap pada package Praktikum1. Buat method menu bertipe void untuk memilih menu program pada saat dijalankan.

Jawaban :

```
sheet10 / Queue13Main.java / Java / Queue13Main / menu
package P1Jobsheet10;

public class Queue13Main {
    public static void menu() {
        System.out.println(x: "Masukkan operasi yang diinginkan");
        System.out.println(x: "1. Enqueue");
        System.out.println(x: "2. Dequeue");
        System.out.println(x: "3. Print");
        System.out.println(x: "4. Peek");
        System.out.println(x: "Clear");
        System.out.println(x: "-----");
    }
}
```

11. Buat fungsi main, kemudian deklarasikan Scanner dengan nama sc.

Jawaban :

```
-----  
  
import java.util.Scanner;  
public class Queue13Main {  
    Run | Debug | Run main | Debug main  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);
```

12. Buat variabel n untuk menampung masukan berupa jumlah maksimal elemen yang dapat disimpan pada queue.

Jawaban :

```
    System.out.print(s: "Masukkan kapasitas queue: ");  
} int n = sc.nextInt();|
```

13. Lakukan instansiasi objek Queue dengan nama Q dengan mengirimkan parameter n sebagai kapasitas elemen queue

Jawaban :

```
    Queue13 Q = new Queue13(n);  
}
```

14. Deklarasikan variabel dengan nama pilih bertipe integer untuk menampung pilih menu dari pengguna.

Jawaban :

```
    ~  
    ~  
    int pilih;|
```

15. Lakukan perulangan menggunakan do-while untuk menjalankan program secara terus menerus sesuai masukan yang diberikan. Di dalam perulangan tersebut, terdapat pemilihan kondisi menggunakan switch-case untuk menjalankan operasi queue sesuai dengan masukan pengguna.

Jawaban :

```

14         do {
15             menu();
16             pilih = sc.nextInt();
17             switch (pilih) {
18                 case 1:
19                     System.out.print(s: "Masukkan data baru: ");
20                     int dataMasuk = sc.nextInt();
21                     Q.enqueue(dataMasuk);
22                     break;
23                 case 2:
24                     int dataKeluar = Q.Dequeue();
25                     if (dataKeluar != 0) {
26                         System.out.println("Data yang dikeluarkan: " + dataKeluar);
27                         break;
28                     }
29                 case 3:
30                     Q.print();
31                     break;
32                 case 4:
33                     Q.peek();
34                     break;
35                 case 5:
36                     Q.clear();
37                     break;
38             }
39         }
40         while (pilih == 1 || pilih == 2 || pilih == 3 || pilih == 4 || pilih == 5);
41     }

```

16. Compile dan jalankan class QueueMain, kemudian amati hasilnya.

2.1.2. Verifikasi Hasil Percobaan

```

Masukkan kapasitas queue: 4
Masukkan operasi yang diinginkan
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru: 15
Masukkan operasi yang diinginkan
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
1
Masukkan data baru: 31

```

```

Masukkan operasi yang diinginkan
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----
4
Elemen terdepan: 15
Masukkan operasi yang diinginkan
1. Enqueue
2. Dequeue
3. Print
4. Peek
5. Clear
-----

```

2.1.3. Pertanyaan

1. Pada konstruktor, mengapa nilai awal atribut front dan rear bernilai -1, sementara atribut size bernilai 0?

Jawaban : karena nilai -1 itu menandakan kalau Queue masih di dalam keadaan kosong. Indeks array dimulai dari 0, jika diinisialisasi dengan 0. Komputer bakal mengira sudah ada data pada indeks ke-0, padahal antrean belum diisi

2. Pada method Enqueue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (rear == max - 1) {  
    rear = 0;
```

Jawaban : maksud dari kode di atas adalah kode ini menerapkan konsep yaitu Circular Queue, yang dimana posisi rear sudah mencapai batas akhir array (max -1). Namun antrean secara keseluruhan belum penuh dikarenakan adanya data depan yang sudah di-dequeue, maka posisi rear bakal diputar kembali ke indeks 0 buat memanfaatkan ruang kosong tersebut.

3. Pada method Dequeue, jelaskan maksud dan kegunaan dari potongan kode berikut!

```
if (front == max - 1) {  
    front = 0;
```

Jawaban : maksud dari kode tersebut pada dequeue sama dengan metode enqueue, merupakan logika circular queue. Ketika data pada indeks terakhir (max -1) diambil dari di-dequeue, maka penunjuk elemen terdepan(front) harus bergeser karena melingkar. Posisi front selanjutnya akan berputar kembali ke indeks 0.

4. Pada method print, mengapa pada proses perulangan variabel i tidak dimulai dari 0 (int i=0), melainkan int i=front?

Jawaban : karena pada queue, data terdepan yang valid tidak selalu berada di indeks 0. Posisi data yang depan siap diproses ditunjukkan oleh front. Oleh karena itu pencetakan data harus dimulai dari posisi front hingga melingkar menuju posisi rear.

5. Perhatikan kembali method print, jelaskan maksud dari potongan kode berikut!

```
i = (i + 1) % max;
```

Jawaban : maksud dari kode di atas adalah rumus operasi sisa hasil bagi (modulu %) dipakai buat menggeser indeks variabel i secara melingkar. Ketika nilai i mencapai max -1, operasi (i + 1) % max akan menghasilkan nilai 0.

6. Tunjukkan potongan kode program yang merupakan queue overflow!

Jawaban :

```
i = (i + 1) % max;
```

7. Pada saat terjadi queue overflow dan queue underflow, program tersebut tetap dapat berjalan dan hanya menampilkan teks informasi. Lakukan modifikasi program sehingga pada saat terjadi queue overflow dan queue underflow, program dihentikan!

Jawaban :

```
public void Enqueue(int dt) {  
    if (isFull()) {  
        System.out.println(x: "Queue sudah penuh");  
        System.out.println(x: "Program dihentikan secara otomatis!");  
        System.exit(status: 0); // menghentikan program  
    }  
    // ...  
}  
  
public int Dequeue() {  
    int dt = 0;  
    if (isEmpty()) {  
        System.out.println(x: "Queue masih kosong");  
        System.out.println(x: "Program dihentikan secara otomatis!");  
        System.exit(status: 0); // menghentikan program  
    }  
    // ...  
}
```

Queue Overflow terjadi ketika kita memaksa memasukkan data (Enqueue) padahal kapasitas array sudah penuh.

Queue Underflow terjadi ketika kita memaksa mengambil data (Dequeue) padahal antrean sudah kosong melongpong.

Angka 0 pada `system.exit(0)` merupakan status code yang memberikan tahu sistem operasi bahwa program berhenti secara normal.

2.2. Percobaan 2 : Antrian Layanan Akademik

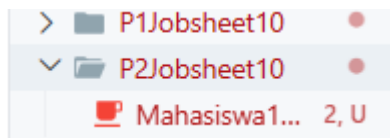
Waktu percobaan : 90 menit

Pada percobaan ini, kita akan membuat program yang mengilustrasikan Layanan pada Admin Akademik.

2.2.1. Langkah-langkah Percobaan

1. Buat folder baru bernama P2Jobsheet10 di dalam repository Praktikum ASD, kemudian buat class baru dengan nama Mahasiswa.

Jawaban :



2. Tambahkan atribut-atribut Nasabah seperti pada Class Diagram, kemudian tambahkan pula konstruktornya seperti gambar berikut ini.

```
public Mahasiswa(String nim, String nama, String prodi, String kelas) {  
    this.nim = nim;  
    this.nama = nama;  
    this.prodi = prodi;  
    this.kelas = kelas;  
}
```

Dan tambahkan method tampilkanData berikut :

```
public void tampilkanData() {  
    System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);  
}
```

Jawaban :

```

2
3 public class Mahasiswa13 {
4     String nim;
5     String nama;
6     String prodi;
7     String kelas;
8
9     public Mahasiswa13(String nim, String nama, String prodi, String kelas) {
10         this.nim = nim;
11         this.nama = nama;
12         this.prodi = prodi;
13         this.kelas = kelas;
14     }
15     public void tampilkanData() {
16         System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
17     }

```

3. Salin kode program class Queue pada Praktikum 1 untuk digunakan kembali pada Praktikum 2 ini, ganti nama class-nya dengan AntrianLayanan. Karena pada Praktikum 1, data yang disimpan pada queue hanya berupa array bertipe integer, sedangkan pada Praktikum 2 data yang digunakan adalah object, maka perlu dilakukan modifikasi pada class AntrianLayanan tersebut.

Jawaban :

```

1  package P2Jobsheet10;
2
3  public class AntrianLayanan13 {
4      Mahasiswa13[] data; //Array menampung Objek Mahasiswa
5      int front;
6      int rear;
7      int size;
8      int max;
9
10     // Constructor Antrean
11     public AntrianLayanan13(int max) {
12         this.max = max;
13         this.data = new Mahasiswa13[max];
14         this.front = 0;
15         this.rear = -1;
16         this.size = 0;
17     }
18
19     public boolean isEmpty() {
20         if (size == 0) {
21             return true;
22         } else {
23             return false;
24         }
25     }
26
27     public boolean isFull() {
28         if (size == max) {
29             return true;
30         } else {
31             return false;
32         }
33     }
34
35     // Mengintip data mahasiswa paling depan di antrean
36     public void peek() {
37         if (!isEmpty()) {
38             System.out.println(x: "Elemen terdepan: ");
39             System.out.println("NIM: " + data[front].nim + " | Nama: " + data[front].nama + " | Prodi: " + data[front].prodi +
40                 " | Kelas: " + data[front].kelas);
41         } else {
42             System.out.println(x: "Queue masih kosong");
43         }
44     }
45

```

```

3  public class AntrianLayanan13 {
44
45     // Mencetak seluruh daftar mahasiswa di dalam antrean saat ini
46     public void print() {
47         if (isEmpty()) {
48             System.out.println(x: "Queue masih kosong");
49         } else {
50             int i = front;
51             while (true) {
52                 System.out.println("NIM: " + data[i].nim + " | Nama: " + data[i].nama + " | Prodi: " + data[i].prodi +
53                     " | Kelas: " + data[i].kelas);
54
55                 if (i == rear) {
56                     break;
57                 }
58                 i = (i + 1) % max;
59             }
60         }
61         System.out.println("Jumlah elemen = " + size);
62     }
63
64     // Mengosongkan total antrean
65     public void clear() {
66         if (!isEmpty()) {
67             front = rear = -1;
68             size = 0;
69             System.out.println(x: "Queue berhasil dikosongkan");
70         } else {
71             System.out.println(x: "Queue masih kosong");
72         }
73     }
74
75

```

```

75 // Menambah mahasiswa ke barisan antrean belakang
76 public void Enqueue(Mahasiswa13 dt) {
77     if (isFull()) {
78         System.out.println(x: "Queue sudah penuh");
79     } else {
80         if (isEmpty()) {
81             front = rear = 0;
82         } else {
83             if (rear == max - 1) {
84                 rear = 0;
85             } else {
86                 rear++;
87             }
88         }
89         data[rear] = dt;
90         size++;
91     }
92 }
93
94 public Mahasiswa13 Dequeue() {
95     Mahasiswa13 dt = null;
96     if (isEmpty()) {
97         System.out.println(x: "Queue masih kosong");
98     } else {
99         dt = data[front];
100         size--;
101         if (isEmpty()) {
102             front = 0;
103             rear = -1;
104         } else {
105             if (front == max - 1) {
106                 front = 0;
107             } else {
108                 front++;
109             }
110         }
111     }
112     return dt;
113 }
114 }

```

4. Lakukan modifikasi pada class AntrianLayanan dengan mengubah tipe int[] data menjadi Mahasiswa[] data karena pada kasus ini data yang akan disimpan berupa object Mahasiswa. Modifikasi perlu dilakukan pada atribut, method Enqueue, dan method Dequeue.

Jawaban :

```
// menambah antrian mahasiswa
public void tambahAntrian13(Mahasiswa13 mhs) {
    if (isFull()) {
        System.out.println(x: "Antrian penuh, tidak dapat menambah mahasiswa.");
        return;
    }
    rear = (rear + 1) % max;
    data[rear] = mhs;
    size++;
    System.out.println(mhs.nama + "Berhasil masuk ke antrian");
}

public Mahasiswa13 layaniMahasiswa13() {
    if (isEmpty()) {
        System.out.println(x: "Antrian kosong.");
        return null;
    }
    Mahasiswa13 mhs = data[front];
    front = (front + 1) % max;
    size--;
    return mhs;
}
```

5. Berikutnya method peek dan print yaitu untuk menampilkan data antrian layanan paling depan dan menampilkan semua data antrian layanan.

Jawaban :

```
public void lihatTerdepan() {
    if (isEmpty()) {
        System.out.println(x: "Antrian kosong.");
    } else {
        System.out.print(s: "Mahasiswa terdepan: ");
        System.out.println(x: "NIM - NAMA - PRODI - KELAS");
        data[front].tampilkanData();
    }
}

public void tampilkanSemua() {
    if (isEmpty()) {
        System.out.println(x: "Antrian kosong.");
        return;
    }
    System.out.println(x: "Daftar Mahasiswa dalam antrian:");
    System.out.println(x: "NIM - NAMA - PRODI - KELAS");
    for (int i = 0; i < size; i++) {
        int index = (front + i) % max;
        System.out.print((i + 1) + ". ");
        data[index].tampilkanData();
    }
}
```

ipilkan nilai size

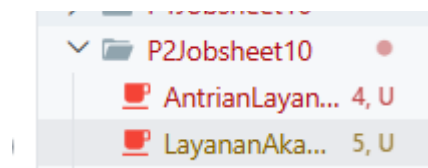
```

    public int getJumlahAntrian() {
        return size;
    }

```

6. Selanjutnya, buat class baru dengan nama LayananAkademikSIKAD tetap pada package yang sama. Buat fungsi main, deklarasikan Scanner dengan nama sc.

Jawaban :



```

import java.util.Scanner;
public class LayananAkademikSIKAD13 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
    }
}

```

7. Kemudian lakukan instansiasi objek AntrianLayanan dengan nama antrian dan nilai parameternya adalah nilai maksimal antrian yang ditentukan (misal sama dengan 5).

Jawaban :

```

AntrianLayanan13 antrian = new AntrianLayanan13(max: 5);

```

8. Deklarasikan variabel dengan nama pilihan bertipe integer untuk menampung pilih menu dari pengguna.

```

public class LayananAkademikSIKAD {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan antrian = new AntrianLayanan(5);
        int pilihan;
    }
}

```

Jawaban :

```

import java.util.Scanner;
public class LayananAkademikSIKAD13 {
    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        AntrianLayanan13 antrian = new AntrianLayanan13(max: 5);
        int pilihan;
    }
}

```

9. Tambahkan kode berikut untuk melakukan perulangan menu sesuai dengan masukan yang diberikan oleh pengguna.

Jawaban :

```

do {
    System.out.println(x: "\n=== Menu Antrian Layanan Akademik ===");
    System.out.println(x: "1. Tambah Mahasiswa ke Antrian");
    System.out.println(x: "2. Layani Mahasiswa");
    System.out.println(x: "3. Lihat Mahasiswa Terdepan");
    System.out.println(x: "4. Lihat Semua Antrian");
    System.out.println(x: "5. Jumlah Mahasiswa dalam Antrian");
    System.out.println(x: "0. Keluar");
    System.out.println(x: "Pilih menu: ");
    pilihan = sc.nextInt(); sc.nextLine();

    switch (pilihan) {
        case 1:
            System.out.print(s: "NIM : ");
            String nim = sc.nextLine();
            System.out.println(x: "Nama : ");
            String nama = sc.nextLine();
            System.out.println(x: "Prodi : ");
            String prodi = sc.nextLine();
            System.out.println(x: "Kelas : ");
            String kelas = sc.nextLine();
            Mahasiswa13 mhs = new Mahasiswa13(nim, nama, prodi, kelas);
            antrian.tambahAntrian13(mhs);
            break;
        case 2:
            Mahasiswa13 dilayani = antrian.layaniMahasiswa13();
            if (dilayani != null) {
                System.out.print(s: "Melayani mahasiswa: ");
                dilayani.tampilkanData();
            }
            break;
        case 3:
            antrian.lihatTerdepan();
            break;
        case 4:
            antrian.tampilkanSemua();
            break;
        case 5:
            System.out.println("Jumlah dalam antrian: " + antrian.getJumlahAntrian());
            break;
        case 0:
            System.out.println(x: "Terima kasih.");
            break;
        default:
            System.out.println(x: "Pilihan tidak valid");
    }
}while (pilihan != 0);

sc.close();
}

```


10. Compile dan jalankan class LayananAkademikSIKAD, kemudian amati hasilnya.

2.2.2 Verifikasi Hasil Percobaan

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM : 123
Nama : Aldi
Prodi : TI
Kelas : 1A
AldiBerhasil masuk ke antrian
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 1
NIM : 124
Nama : Bobi
Prodi : TI
Kelas : 1G
BobiBerhasil masuk ke antrian
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam antrian:
NIM - NAMA - PRODI - KELAS
1. 123 - Aldi - TI - 1A
2. 124 - Bobi - TI - 1G
Jumlah dalam antrian: 2
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 2
Melayani mahasiswa: 123 - Aldi - TI - 1A
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 4
Daftar Mahasiswa dalam antrian:
NIM - NAMA - PRODI - KELAS
1. 124 - Bobi - TI - 1G
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 5
Jumlah dalam antrian: 1
```

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
0. Keluar
Pilih menu: 0
```

2.2.3 Pertanyaan

Lakukan modifikasi program dengan menambahkan method baru bernama LihatAkhir pada class AntrianLayanan yang digunakan untuk mengecek antrian yang berada di posisi belakang. Tambahkan pula daftar menu 6. Cek Antrian paling belakang pada class LayananAkademikSIKAD sehingga method LihatAkhir dapat dipanggil!

Jawaban :

Modifikasi pada class Antrian layanan menampilkan lihat akhir

```
// modifikasi lihatAkhir
public void lihatAkhir() {
    if (isEmpty()) {
        System.out.println(x: "Antrian palilng belakang: ");
        data[rear].tampilkanData();
    } else {
        System.out.println(x: "Antrian masih kosong!");
    }
}
```

Modifikasi Class LayananAkademikSIKAD

Menambahkan case lihat akhir :

```
        break;
    case 6:
        antrian.lihatAkhir();
        break;
```

Menambah opsi menu lihat akhir :

```
System.out.println(x: "5. Jumlah Mahasiswa dalam Antrian");
System.out.println(x: "6. Cek Antrian paling belakang");
```

Output :

```
=== Menu Antrian Layanan Akademik ===
1. Tambah Mahasiswa ke Antrian
2. Layani Mahasiswa
3. Lihat Mahasiswa Terdepan
4. Lihat Semua Antrian
5. Jumlah Mahasiswa dalam Antrian
6. Cek Antrian paling belakang
0. Keluar
Pilih menu: 6
Antrian palilng belakang:
RURY - 123 - TI - 10
```

2.3 Tugas

Buatlah program antrian untuk mengilustrasikan antrian persetujuan Kartu Rencana Studi (KRS) Mahasiswa oleh Dosen Pembina Akademik (DPA). Ketika seorang mahasiswa akan mengantri, maka dia harus mendaftarkan datanya (data mahasiswa seperti pada praktikum 2). Gunakan class untuk antrian seperti pada Praktikum 1 dan 2, dengan method-method yang berfungsi :

- Cek antrian kosong, Cek antrian penuh, Mengosongkan antrian.
 - Menambahkan antrian, Memanggil antrian untuk proses KRS – setiap 1x panggilan terdiri dari 2 mahasiswa (pada antrian no 1 dan 2)
 - Menampilkan semua antrian, Menampilkan 2 antrian terdepan, Menampilkan antrian paling akhir.
 - Cetak jumlah antrian, Cetak jumlah yang sudah melakukan proses KRS
 - Jumlah antrian maximal 10, jumlah yang ditangani masing-masing DPA 30 mahasiswa, cetak jumlah mahasiswa yang belum melakukan proses KRS.
- Gambarkan Diagram Class untuk antriannya. Implementasikan semua method menggunakan menu pilihan pada fungsi main.

Jawaban :

```
Mahasiswa13.java U × QueueKRS13.java 2, U MainKRS13.java 4, U
T1Jobsheet10 > Mahasiswa13.java > Java > Mahasiswa13 > tampilkanData()

1 package T1Jobsheet10;
2
3 public class Mahasiswa13 {
4     String nim;
5     String nama;
6     String prodi;
7     String kelas;
8
9     public Mahasiswa13(String nim, String nama, String prodi, String kelas) {
10         this.nim = nim;
11         this.nama = nama;
12         this.prodi = prodi;
13         this.kelas = kelas;
14     }
15
16     public void tampilkanData() {
17         System.out.println(nim + " - " + nama + " - " + prodi + " - " + kelas);
18     }
19
20 }
```

```
Mahasiswa13.java U QueueKRS13.java 2, U × MainKRS13.java 4, U
T1Jobsheet10 > QueueKRS13.java > Java > QueueKRS13 > panggilKRS()

1 package T1Jobsheet10;
2
3 public class QueueKRS13 {
4     Mahasiswa13[] data;
5     int front;
6     int rear;
7     int size;
8     int max;
9     int sudahKRS;
10    int totalKapasitasDPA = 30;
11
12    public QueueKRS13(int n) {
13        max = n;
14        data = new Mahasiswa13[max];
15        size = 0;
16        front = 0;
17        rear = -1;
18        sudahKRS = 0;
19    }
20 }
```

```

20 public class QueueKSI3 {
21     // Cek antrian kosong
22     public boolean isEmpty() {
23         return size == 0;
24     }
25
26     // cek antrian penuh
27     public boolean isFull() {
28         return size == max;
29     }
30
31     // mengosongkan antrian
32     public void clear() {
33         front = 0;
34         rear = -1;
35         size = 0;
36         System.out.println(x: "Antrian berhasil dikosongkan");
37     }
38
39     // Menambahkan antrian
40     public void tambahAntrian(Mahasiswa13 mhs) {
41         if (!isFull()) {
42             rear = ( rear + 1) % max;
43             data[rear] = mhs;
44             size++;
45             System.out.println("Mahasiswa " + mhs.nama + " berhasil masuk ke antrian");
46         } else {
47             System.out.println(x: "Gagal tambah! Antrian sudah penuh (maks 10).");
48         }
49     }
50 }

```

```

public class QueueKRS13 {
    // Memanggil antrian untuk proses KRS
    public void panggilKRS() {
        if (isEmpty()) {
            System.out.println(x: "Antrian kosong! tidak ada mahasiswa yang di panggil");
            return;
        }

        System.out.println(x: "=== MEMANGGIL MAHASISWA UNTUK PROSES KRSS ===");
        int jumlahDipanggil = (size >= 2) ? 2 : 1;

        for (int i = 0; i < jumlahDipanggil; i++) {
            Mahasiswa13 mhs = data[front];
            System.out.print(s: "Diproses: ");
            mhs.tampilkanData();

            front = (front + 1) % max;
            size --;
            sudahKRS++;
        }

        if (isEmpty()) {
            front = 0;
            rear = -1;
        }
    }
}

```

```

public class QueueKRS13 {
77 // Menampilkan semua antrian
78 public void tampilkanSemua() {
79     if (!isEmpty()) {
80         System.out.println(x: "--- DAFTAR SEMUA ANTRIAN KRS ---");
81         int i = front;
82         int count = 0;
83         while (count < size) {
84             System.out.print((count + 1) + ". ");
85             data[i].tampilkanData();
86             i = (i + 1) % max;
87             count++;
88         }
89     } else {
90         System.out.println(x: "Antrian kosong");
91     }
92 }
93
94 // Menampilkan 2 antrian terdepan
95 public void tampilkanDuaTerdepan() {
96     if (isEmpty()) {
97         System.out.println(x: "Antrian kosong");
98         return;
99     }
100     System.out.println(x: "--- 2 ANTRIAN TERDEPAN ---");
101     System.out.print(s: "1. ");
102     data[front].tampilkanData();
103
104     if (size >= 2) {
105         int indeksKedua = (front + 1) % max;
106         System.out.print(s: "2. ");
107         data[indeksKedua].tampilkanData();
108     }
109 }
...

```

```

// Menampilkan antrian paling akhir
public void tampilkanPalingAkhir() {
    if (!isEmpty()) {
        System.out.println(x: "--- ANTRIAN PALING AKHIR ---");
        data[rear].tampilkanData();
    } else {
        System.out.println(x: "Antrian kosong!");
    }
}

// jumlah antrian saat ini
public void cetakJumlahAntrian() {
    System.out.println("Jumlah mahasiswa dalam antrian saat ini: " + size);
}

// cetak KRS berdasarkan Sudah ada KRS / belum ada KRS berdasarkan kuota DPA)
public void cetakStatistikKRS() {
    int belumKRS = totalKapasitasDPA - sudahKRS;
    System.out.println(x: "--- STATISTIK PROSES KRS OLEH DPA ---");
    System.out.println("Jumlah mahasiswa SUDAH proses KRS : " + sudahKRS);
    System.out.println("Sisa mahasiswa BELUM proses KRS : " + (belumKRS < 0 ? 0 : belumKRS) + " dari kuota 30 mahasiswa");
}

```

 Mahasiswa13.java U

 QueueKRS13.java 2, U

 MainKRS13.java 4, U X

T1Jobsheet10 >  MainKRS13.java > Java >  MainKRS13 >  main(String[] args)

```

1  package T1Jobsheet10;
2
3  import java.util.Scanner;
4  public class MainKRS13 {
5      Run | Debug | Run main | Debug main
6      public static void main(String[] args) {
7          Scanner sc = new Scanner(System.in);
8          QueueKRS13 antrianKRS13 = new QueueKRS13(n: 10);

```

```

>  public static void main(String[] args) {
8
9      int pilihan;
10     do {
11         System.out.println(x: "=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===");
12         System.out.println(x: "1. Tambah Mahasiswa Ke Antrian");
13         System.out.println(x: "2. Panggil Mahasiswa Untuk KRS");
14         System.out.println(x: "3. Lihat Semua Antrian");
15         System.out.println(x: "4. Lihat 2 Antrian Terdepan");
16         System.out.println(x: "5. Lihat Antrian Paling Akhir");
17         System.out.println(x: "6. Cetak Jumlah Baris Saat Ini");
18         System.out.println(x: "7. Cetak Statistik KRS ( sudah / belum KRS)");
19         System.out.println(x: "8. Kosongkan Antrian");
20         System.out.println(x: "0. Keluar Program");
21         System.out.print(s: "Pilih menu: ");
22         pilihan = sc.nextInt();
23         sc.nextLine();

```

```

25 public static void main(String[] args) {
26     switch (pilihan) {
27         case 1:
28             System.out.print(s: "NIM : ");
29             String nim = sc.nextLine();
30             System.out.print(s: "Nama : ");
31             String nama = sc.nextLine();
32             System.out.print(s: "Prodi : ");
33             String prodi = sc.nextLine();
34             System.out.print(s: "Kelas : ");
35             String kelas = sc.nextLine();
36             Mahasiswa13 mhs = new Mahasiswa13(nim, nama, prodi, kelas);
37             antrianKRS13.tambahAntrian(mhs);
38             break;
39         case 2:
40             antrianKRS13.panggilKRS();
41             break;
42         case 3:
43             antrianKRS13.tampilkanSemua();
44             break;
45         case 4:
46             antrianKRS13.tampilkanDuaTerdepan();
47             break;
48         case 5:
49             antrianKRS13.tampilkanPalingAkhir();
50             break;
51         case 6:
52             antrianKRS13.cetakJumlahAntrian();
53             break;
54         case 7:
55             antrianKRS13.cetakStatistikKRS();
56             break;
57         case 8:
58             antrianKRS13.clear();
59             break;
60         case 0:
61             System.out.println(x: "Program selesai. Terima kasih");
62             break;
63         default:
64             System.out.println(x: "Pilihan tidak valid! Silahkan pilih menu kembali");
65     } while (pilihan !=0);
66 }

```

1.Tampilan Awal & Menambahkan Antrean (Menu 1)

```

KRS13'
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 1
NIM : 123
Nama : Barbie
Prodi : TI
Kelas : 1A
Mahasiswa Barbie berhasil masuk ke antrian

```

```

Mahasiswa Barbie berhasil masuk ke antrian
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 1
NIM : 124
Nama : Barbai
Prodi : TI
Kelas : 1D
Mahasiswa Barbai berhasil masuk ke antrian

```

```

Mahasiswa Barbai berhasil masuk ke antrian
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 1
NIM : 125
Nama : Racel
Prodi : TI
Kelas : 1C
Mahasiswa Racel berhasil masuk ke antrian

```


2. Melihat Semua Antrean Saat Ini (Menu 3)

```
manasiswa racel bernasi masuk ke antrian
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 3
--- DAFTAR SEMUA ANTRIAN KRS ---
1. 123 - Barbie - TI - 1A
2. 124 - Barbai - TI - 1D
3. 125 - Racel - TI - 1C
```

3. Melihat 2 Antrean Terdepan & Paling Akhir (Menu 4 & Menu 5)

```
3. 125 - Racel - TI - 1C
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 4
--- 2 ANTRIAN TERDEPAN ---
1. 123 - Barbie - TI - 1A
2. 124 - Barbai - TI - 1D
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 5
--- ANTRIAN PALING AKHIR ---
125 - Racel - TI - 1C
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
```

4. Memanggil Mahasiswa untuk Proses KRS (Menu 2)

```
125 - Racel - TI - 1C
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 2
=== MEMANGGIL MAHASISWA UNTUK PROSES KRSS ===
Diproses: 123 - Barbie - TI - 1A
Diproses: 124 - Barbai - TI - 1D
```

5. Cek Jumlah Baris & Statistik KRS Saat Ini (Menu 6 & Menu 7)

```
Diproses: 124 - Barbai - TI - 1D
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 6
Jumlah mahasiswa dalam antrian saat ini: 1
=== SISTEM ANTRIAN PERSETUJUAN KRS OLEH DPA ===
1. Tambah Mahasiswa Ke Antrian
2. Panggil Mahasiswa Untuk KRS
3. Lihat Semua Antrian
4. Lihat 2 Antrian Terdepan
5. Lihat Antrian Paling Akhir
6. Cetak Jumlah Baris Saat Ini
7. Cetak Statistik KRS ( sudah / belum KRS)
8. Kosongkan Antrian
0. Keluar Program
Pilih menu: 7
--- STATISTIK PROSES KRS OLEH DPA ---
Jumlah mahasiswa SUDAH proses KRS : 2
Sisa mahasiswa BELUM proses KRS : 28 dari kuota 30 mahasiswa
```